

Case ID	Student Name	LLM Model Name Used	Original Code Snippet	New Code Snippet with Bugs	Your Initial Self Reflection	GenAI's Label (Good or Bad)	GenAI's Guidance	Your New Self Reflection
1	Drake Davies	Gemini	loss = 'binary_crossentropy'	loss = 'categorical_crossentropy'	I think that changing the loss from binary to categorical crossentropy produces bad results, which I would assume comes from the system being built for binary inputs.	Bad	To reason more deeply about this, let's look at the structure of your network and what the new loss function expects. Take a look at the final layer of your model: Dense(1, activation='sigmoid'). How many numbers is this layer actually predicting for a single review? Next, think about the word "categorical". If you are using categorical crossentropy, what kind of output format—and how many output nodes—is that loss function mathematically expecting to evaluate? Finally, consider what your current training labels (the y_train data) look like. Ask yourself how passing a single 0 or 1 label conflicts with the probability distribution that a categorical loss function is trying to compare against.	Because the model uses the sigmoid activation function, reviews are translated into one number between 0 and 1. This doesn't work well with the categorical loss, because categorical requires several or more numbers to evaluate, which is also why the binary crossentropy works so well with the sigmoid function.
2	Drake Davies	Gemini	maxlen = 200	maxlen = 20	I think that decreasing the max length of training sequences from 200 to 20 results in lower accuracy and higher loss because the model doesn't have as much data to work with.	Bad	You've correctly observed that reducing the length of the sequences degrades the model's performance, but your reasoning about the cause is a bit superficial. When you say the model "doesn't have as much data to work with," it sounds like you are suggesting the *number* of examples has decreased. However, you are still training on 25,000 reviews. What has actually changed is the amount of information available *within each individual example*. How does limiting the "window" of each review to such a small size affect the model's ability to "see" the sentiment-carrying words?	I see that reducing the max length from 200 to 20 heavily limits each review that the model is being trained on, cutting out up to 180 words from the reviews. It's possible or likely that the true sentiment of a review will be towards the middle or end, which is now missing because of this change. The model can not accurately train itself using the reviews when only a portion of each review can be examined.
3	Drake Davies	Gemini	loss, accuracy = model.evaluate(x_test, y_test, verbose=0)	loss, accuracy = model.evaluate(x_train, y_train, verbose=0)	I believe that the accuracy and loss are improved by evaluating the training data, because this data is different from the testing data and must be resulting in higher outputs.	Bad	You've noticed an important pattern: the model performs better on the data it was trained on than on data it has never seen before. However, your reflection is a bit superficial because it doesn't explain the underlying machine learning concept at play or why this "improvement" is actually a misleading sign for a developer. What is the technical term for when a model performs significantly better on its training data than on new, unseen data? How does that term relate to your observation?	The accuracy and loss are impacted with this change, because evaluating the data that the model was already trained on will artificially increase it's accuracy and lower its loss. Evaluating through a test rather than on the training data provides a better idea of how accurate the model is, by seeing what is has learned compared to testing it on what it already knows.
4	Drake Davies	Gemini	Embedding (input_dim=10000, output_dim=16, input_length=maxlen)	Embedding (input_dim=10000, output_dim=1, input_length=maxlen)	Decreasing the output_dim lowered accuracy, improved loss, and produced bad review sentiments, and I think it's because the way the model learns was negatively impacted, and thus hurt the accuracy.	Bad	You've correctly observed that decreasing the 'output_dim' negatively impacted the model, but your reasoning is a bit superficial. Saying it was "negatively impacted" describes *that* it happened, but doesn't explain the mechanism of *why* the model is now struggling to distinguish between a "good" and "bad" movie. Ask yourself: If you have to map 10,000 unique words into a 1-dimensional space (essentially a single number line), how much room does the model have to create a meaningful "distance" between words that mean different things? How does this "bottleneck" prevent the model from capturing the nuance of human language?	Decreasing the output_dim hurt the accuracy of the model, because each word the model looks at is represented by only 1 dimension instead of the previous 16. This reduces the uniqueness of each word and minimizes the space between significantly different words, ultimately sabotaging the effectiveness of the model.

Case ID	Student Name	LLM Model Name Used	Original Code Snippet	New Code Snippet with Bugs	Your Initial Self Reflection	GenAI's Label (Good or Bad)	GenAI's Guidance	Your New Self Reflection
5	Drake Davies	Gemini	<code>(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=10000)</code>	<code>(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=5000)</code>	Decreasing the number words in the loading process had a slight negative impact on accuracy and loss, and I believe this happened because the model doesn't have enough or as much data to train itself on.	Bad	While you've correctly noted that the performance dropped, your explanation that the model "doesn't have as much data to train itself on" is a bit superficial. Remember that <code>imdb.load_data</code> still returns the same number of training samples (25,000 reviews). What changed was not the quantity of reviews, but the vocabulary available within them. Word Frequency: In a movie review, which words are likely in the "top 5,000" (like "the", How does reducing the "dictionary" size change the quality of the information the model sees in each review, even if the number of reviews remains the same?	By reducing the number of words from 10000 to 5000, the model has a weaker vocabulary to work with, only having half of the most common words. This creates issues with more broad vocabulary being used in reviews, where the model wouldn't have been trained on more unique words.
6	Drake Davies	Gemini	<code>Dense(1, activation='sigmoid')</code>	<code>Dense(1, activation='relu')</code>	I think changing the activation function in this step from sigmoid to relu allows for a wider range of numbers to be output, which means more variety in score predictions and a lower accuracy and higher loss as well.	Bad	You have correctly identified that 'relu' allows for a wider range of numbers compared to 'sigmoid', but your reflection is a bit superficial because it doesn't address the mathematical "mismatch" between the activation function and the goal of the model. To think more deeply about this, consider the following: 1. The Goal of the Output: This model is performing binary classification (Positive vs. Negative). What is the specific range of values that the 'binary_crossentropy' loss function and your training labels ('0' and '1') expect the model to produce? . The Probability Problem: In machine learning, we often treat the final output as a "probability score." If a review is extremely positive, the Sigmoid function will cap it at a value very close to 1.0. If the ReLU function calculates a very "positive" result, it might output a value like 15.7 or 100.0. How does a loss function react when it expects a probability (0 to 1) but receives a number like 50? How does using a function that has no upper limit (ReLU) create a conflict with a system designed to categorize things into a 0 or 1 range?	The binary crossentropy part of the code expects binary numbers, which aren't provided when changing the activation function from sigmoid to relu. The relu function can output a much broader range of numbers, which doesn't support the binary system. These output numbers also don't fit the constraints of the probability of the score, because they are not restricted between 0 and 1, where this would have happened in the sigmoid function.
7	Drake Davies	Gemini	<code>encoded = [word_index.get(w, 2) + 3 for w in words]</code>	<code>encoded = [word_index.get(w, 2) + 0 for w in words]</code>	By changing the addition here from 3 to 0, the output became inaccurate sentiment analysis, and I think this is because the indexing of words must be messed up, and the model uses the wrong words in its prediction.	Bad	You've noticed that changing the "offset" (the number added to the word index) from 3 to 0 causes the model to fail, and you've correctly guessed that the "indexing of words" is now wrong. However, your explanation is a bit superficial because it doesn't explain *how* the indexing was specifically designed to work in the Keras IMDB dataset. In this dataset, the integers aren't just random IDs; the first few numbers are reserved for special purposes. Look at how the 'reverse_word_index' was constructed in the original code: <code>'decoded_review = ''.join([reverse_word_index.get(i - 3, '?') for i in x_train[0]])</code> Why was that '- 3' there in the first place? What happens to the "dictionary" if you remove that offset?	By changing the indexing of the prediction, the model grabs the wrong words according to the index offsetting occurring previously in the code. The +3 was needed because the word indexing was done using a -3 prior, where there were other uses for these indices, and by removing the +3, the entire system is messed up, affecting the prediction and sentiment provided.

Case ID	Student Name	LLM Model Name Used	Original Code Snippet	New Code Snippet with Bugs	Your Initial Self Reflection	GenAI's Label (Good or Bad)	GenAI's Guidance	Your New Self Reflection
8	Drake Davies	Gemini	from tensorflow.keras.layers import Dense, Embedding, GlobalAveragePooling1D, (LATER IN THE CODE->) GlobalAveragePooling1D()	from tensorflow.keras.layers import Dense, Embedding, GlobalAveragePooling1D, Flatten, (LATER IN THE CODE->) Flatten()	Changing from GlobalAveragePooling1D to Flatten resulted in much lower sentiment scores, despite similar accuracy and loss scores. I think this is because the way the data is handled differently in the intermediate steps must be having an effect on the final predictions.	Bad	You've correctly observed that the behavior of the model's predictions changed, but saying the data is "handled differently" is a bit superficial. To understand why your sentiment scores are suddenly much closer to the middle or "weaker," you need to look at the mathematical difference between **Pooling** and **Flattening**. To think more deeply about this, consider this question: The Math of Aggregation: GlobalAveragePooling1D takes the average of all the word vectors in a review. If a review has 200 words, it squashes them into one "average" representation. Flatten, on the other hand, takes all 200 words and lines them up end-to-end. Which one cares more about the exact position of a word? How does shifting from a "summary" of the words (Average) to a "rigid map" of every single word position (Flatten) change the way the model weights the importance of specific sentiment words?	Using Flatten forces the model to use a more raw, long list of data compared to GlobalAveragePooling, which aggregates the data through averaging. This means the Flatten layer is passing along many more values while GlobalAveragePooling is using less, making the model less confident as it has to work with many more values, pinpointing exact words rather than using averages. This hesitancy is what causes the lower sentiment scores seen in the trial.
9	Drake Davies	Gemini	validation_split=0.2	validation_split=0.0	There is a key error when running the validation split at 0 and the code doesn't finish running, and I think it's because the model doesn't have any data to use as validation and thus can't verify its results	Bad	While you've correctly identified that the model lacks validation data, your reflection is a bit superficial because it doesn't address why the *entire program* crashes with a "Key Error" rather than just giving you a poor result. To think more deeply about this, look further down in your code—past the <code>'model.fit()'</code> function. Think about the relationship between the parameters you set in <code>'model.fit'</code> and the visualization code that follows. Why does the graphing part of your script specifically depend on that <code>'validation_split'</code> being greater than zero?	The key error occurs because later in the code, the model uses validation data to plot and compare to training data. Setting the validation split to 0 removes this validation data, and therefore when the code calls to plot the validation data, it results in an error.
10	Drake Davies	Gemini	[previously blank, above history = model.fit()]	np.random.shuffle(y_train)	Shuffling the y_train data right before testing significantly lowered accuracy and increased loss, also resulting in incorrect sentiment analysis results. I think randomizing the data makes it too difficult for the model to assess the reviews, resulting in inaccuracies across the board.	Bad	You've correctly identified that the randomization is the cause of the poor performance, but your reasoning is a bit superficial. Saying that randomizing the data makes it "too difficult" for the model suggests that the task itself became harder, rather than identifying a fundamental break in the logic of the training process. Think about what the model is actually seeing now. Is it seeing a "difficult" pattern, or is it being forced to learn that a positive review text is actually "negative" (and vice-versa) because the labels no longer belong to their original stories?	Because I shuffled the order of the training data, the labels of good or bad reviews are no longer properly associated with their reviews, so it's unlikely for the model to accurately assess these in testing, on a consistent basis.